SPRING AI INCIDENT RESPONDER

SECTION 6 | LECTURE 14

수업 14. 여러 LLM 호출에 정책을 어떻게 일관되게 적용할까?

Advisor로 공통 정책 적용하기

⚠ 일관성 결핍 식별: 모든 LLM 호출에 정책이 전달되고 있는가?

CALL 01

Incident Analysis

장애 원인 및 상황 분석 Prompt에 정책 정보 직접 결합

`Policy Context: YES`

CALL 02

Action Recommendation

추천 Action 생성 Prompt에 에스컬레이션 정책 직접 결합

`Policy Context: YES`

CALL 03

Quality Evaluation

Evaluator는 의미 품질 4축만 평가할 뿐 공통 정책 문맥 결여

`Policy Context: MISSING`

CALL 04

Quality Optimization

Optimizer가 출력을 재작성할 때 공통 정책 제약 문맥 미전달

`Policy Context: MISSING`

01. DUPLICATION 정책 코드 중복

여러 Prompt마다 동일한 정책 조립 로직을 복사·붙여넣기하며 코드 팽창

02. MISSING CONTEXT 정책 전달 누락

새로운 LLM 호출이 추가될 때마다 정책 컨텍스트 주입이 누락되어 위험 노출

03. INCONSISTENCY 표현 불일치

시간이 지나며 호출 지점별로 정책 텍스트 표현이 미세하게 갈라져 혼선 초래

? **핵심 문제:** 정책이 존재한다고 해서 **모든 LLM 호출에 자동으로 전달되지는 않습니다.**

`Policy Exists ≠ Uniform Delivery`

호출 경계 통제: 공통 정책 컨텍스트 일관 주입

// ARCHITECTURAL BOUNDARY: POLICY INJECTION PIPELINE

Java Decision → Advisor Boundary → LLM Inference

JAVA DOMAIN **Application Policy Engine** (minimumSeverity 대조 · EscalationDecision 객체 빌드) [정책 판단 권한 유지]

↓ ChatClientRequest Context 파라미터 전달

CALL BOUNDARY **IncidentPolicyAdvisor (Client Request Mutation)**(공통 정책 컨텍스트를 Prompt 경계에서 가로채 주입) [일관된 주입 보장]

↓ 모든 LLM 호출에 동일 정책 컨텍스트 동시 전파

Analysis Call | Action Call | Evaluation Call | Optimization Call

ROLE 01. JAVA APP **정책 판단 권한**

minimumSeverity 및 에스컬레이션 규칙에 따라 정책을 결정 (Advisor에 위임 불가)

ROLE 02. ADVISOR **정책 컨텍스트 전달**

이미 결정된 구조화 정책을 가로채 모든 LLM 호출 Prompt에 누락 없이 일관되게 결합

ROLE 03. POLICY GUARD **최종 응답 정책 적용**

Prompt 전달은 참고용이며, 최종 Action 위험도 분류와 노출 여부는 Java가 직접 통제

통제 원칙: 정책 판단은 **Java 애플리케이션**이 수행하고, 공통 컨텍스트 주입은 **Advisor**가 담당합니다.

* Separation of Policy Decision and Context Injection

실습 검증 시나리오: 오늘 실습에서 확인할 것

STEP 01. IDE **공통 정책 적용 지점 구성**

Prompt마다 정책을 직접 추가하던 구조를 제거하고, Advisor를 통해 공통 정책 컨텍스트를 전달하는 구조 확인

STEP 02. UNIT TEST **Advisor 컨텍스트 검증**

정책 정보가 있으면 기존 Prompt에 공통 정책 컨텍스트가 추가되고, 없으면 기존 Prompt가 그대로 유지되는지 단위 테스트로 검증

STEP 03. POSTMAN **전체 Incident 흐름 정상 동작 확인**

Advisor 적용 이후에도 Incident 요청부터 분석, 추천, 품질 통제, 최종 응답까지 전체 처리 흐름이 정상적으로 동작하는지 확인

// ADVISOR POLICY INTEGRATION VERIFICATION TARGETS

Consistent Call Boundary Enforcement

Request Context Augmentation	기존 Prompt 훼손 없이 에스컬레이션 정책 컨텍스트만 안전 결합	[단위 테스트 검증]
Missing Context Safe Pass-through	정책 파라미터 부재 시 기존 요청 원형 그대로 통과	[단위 테스트 검증]
End-to-End Pipeline Health	Analysis/Action/Quality 등 복수 호출 파이프라인 무결성 유지	[Postman 응답 확인]

⚠ **실습 핵심:** 정책을 새로 판단하는 것이 아니라, **이미 결정된 정책을 여러 LLM 호출에 일관되게 전달하는 구조**를 확인합니다.

* IDE 실습 전환 준비

오늘 정리 & 다음 수업 안내

TODAY SUMMARY

Policy Advisor

정책 판단과 컨텍스트 전달의 책임 분리

정책 판단 책임: minimum Severity 및 에스컬레이션 결정은 Java 애플리케이션이 수행

컨텍스트 전달 책임: 결정된 공통 정책 문맥은 Advisor가 LLM 호출 경계에서 일관 주입

최종 정책 적용: Prompt 전달은 참고용이며 최종 Action 위험 통제는 Java가 직접 수행

누락·중복 방지: Analysis, Action, Evaluation, Optimization 전반에 일관성있는 적용 달성

통제 결론: 정책 판단은 Java가, 공통 정책 컨텍스트 전달은 Advisor가, **최종 정책 적용은 다시 Java 애플리케이션이 담당합니다.**

UP NEXT LECTURE

다음 수업 미리보기 (복합 위험 검토와 패턴 도입 판단)

공통 정책은 여러 LLM 호출에 일관되게 전달됩니다. 그렇다면 위험 검토가 여러 관점으로 복잡해질 때, 기존 구조로 충분할까요? 새로운 패턴이 필요할까요?

Lesson 14 (정책 컨텍스트): 공통 정책 컨텍스트를 여러 LLM 호출에 어떻게 일관되게 전달할 것인가?

Next Lesson (패턴 도입 판단): 복합 위험 검토에 Orchestrator–Workers가 필요한 경우는 언제인가?

TRANSITION: POLICY CONTEXT → ARCHITECTURE DECISION 기존 구조와 Orchestrator–Workers를 비교하고 도입 필요성 판단하기

Move to Orchestrator–Workers